

Esame OS Internals (Cabodi)

Data: 11/07/2024

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale e Paginazione)

Contesto: Un sistema informatico ha le seguenti caratteristiche:

- Dimensione della memoria fisica: 512 MB
- Spazio degli indirizzi virtuali: indirizzi a 32 bit
- Dimensioni pagina: 8 KB
- Utilizza una tabella di pagine a due livelli
- Implementa un algoritmo di sostituzione delle pagine least recently used (LRU)

Q1.A) Rispondere alle domande seguenti (Vero/Falso o Scelta Multipla):

1. La presenza di una TLB garantisce sempre un tempo di accesso alla memoria più rapido rispetto a un sistema senza TLB?
2. È possibile eseguire il mapping di due indirizzi virtuali diversi allo stesso indirizzo fisico in un sistema che utilizza il paging?
3. A un processo sono allocati più frame fisici rispetto alle dimensioni del working set (per un determinato tempo delta): questo ridurrà sempre il tasso di errore della pagina a zero?
4. È possibile progettare una tabella di pagine dimensionata dinamicamente in base al numero di pagine attualmente utilizzate da un processo, riducendo così al minimo il carico di memoria?
5. L'utilizzo di una pagina di dimensioni maggiori può comportare un utilizzo più efficiente della TLB nonostante il potenziale aumento della frammentazione interna?
6. Se viene utilizzato l'algoritmo di sostituzione pagine LRU, garantisce il minor numero possibile di page fault?

(Nota: Per ognuna delle risposte alle domande precedenti, è richiesta una motivazione/spiegazione).

Q1.B) Esercizio LRU:

Data la seguente stringa di riferimento a memoria per un processo (indirizzamento dei byte, con indirizzi espressi in codice esadecimale) e le rispettive operazioni di lettura(R)/scrittura (W):

R 1F40, W 3E20, R 5D18, W 7C9E, R 2AF3, W 6B2C, R 13B2, W 24AB, R 54BC, W 12AF

Si suppongano 3 frame disponibili e si utilizzi l'algoritmo di sostituzione della pagina LRU.

1. Calcolare la stringa di riferimenti di pagina e indicare il numero di pagina corrispondente per ogni riferimento a memoria.

2. Indicare esplicitamente per ogni frame allocato e tempo/riferimento, il numero di pagina e se si verifica un page fault.
 3. Calcolare il numero totale di page fault che si verificano durante questa sequenza di riferimenti alla memoria.
-

Domanda 2 (File System e Inodes)

Contesto: Consider a Unix-like file system based on inodes, with 15 pointers (12 direct, 1 single indirect, 1 double indirect, and 1 triple indirect). Pointers have a size of 32 bits, and disk blocks have a size of 4 KB. The file system resides on a disk partition where 1 TB is reserved for data blocks. The extra space reserved for metadata (including index blocks) can be neglected for the purposes of this exercise.

- Q2.A)** Calculate the maximum file size supported by this file system.
- Q2.A)** Given a binary file of size 25,600 KB, compute exactly how many index blocks and data blocks the file is using. Also compute the internal fragmentation (for data blocks).
- Q2.A)** Suppose an operation `lseek(fd, offset, SEEK_END)` is called to position the file offset for a subsequent read/write operation. Given a binary file of size 10 GB and an offset of -1,048,576 bytes (1 MB) from the end of the file, compute the logical block number (relative to the file blocks, numbered starting at 0) to which the position is moved.
-

Domanda 3 (I/O, HDD, SSD e RAID)

Contesto: Si consideri un sistema informatico che utilizza una combinazione di unità disco rigido (HDD) e unità a stato solido (SSD) per l'archiviazione di massa. Il sistema incorpora anche vari dispositivi di I/O gestiti da un sottosistema di I/O.

Q3.A) Rispondere alle seguenti domande (e fornire spiegazioni/motivazioni):

1. È sempre più efficiente utilizzare l'accesso diretto alla memoria (DMA) per tutti i tipi di operazioni di I/O rispetto all'I/O basato su interrupt?
2. Il polling è un metodo più efficiente per la gestione delle operazioni di I/O rispetto all'utilizzo di interrupt in un ambiente di rete ad alta velocità?
3. È possibile che un'operazione di I/O su un SSD abbia sempre una latenza inferiore rispetto alla stessa operazione su un HDD?
4. È possibile che un SSD soffra di problemi di frammentazione simili a quelli riscontrati dagli HDD?
5. Uno scheduler I/O ben progettato può migliorare significativamente le prestazioni di una configurazione RAID 5 rispetto a uno scheduler I/O mal progettato? (*RAID 5: block interleaved distributed parity, è caratterizzato da bit di parità ridondanti*)
6. Una configurazione RAID 0 (striping) può migliorare l'affidabilità dei dati rispetto a una configurazione HDD singolo?

2 SECTION 2: OS161

Domanda 4 (Indirizzamento OS161 e `load_elf`)

Contesto: Viene fornito un sistema OS161, in esecuzione su un simulatore MIPS `sys161` con 8 MB di memoria RAM. È noto che il processo P1 ha uno spazio di indirizzamento con `as->as_pbase1`, `as->as_pbase2`, `as->as_vbase1`, `as->as_vbase2`, `as->as_npages1`, `as->as_npages2` con i seguenti valori: 0x200000, 0x100000, 0x4000, 0x8000, 3, 4.

Q4.A) Traduzione degli indirizzi:

Per ognuno dei seguenti indirizzi fisici, convertire in un indirizzo logico del kernel. Nel caso in cui l'indirizzo sia allocato a P1, convertirlo anche in un indirizzo user logico:

1. 0x100A00
2. 0x10A0F0
3. 0x206500
4. 0x202D00

Q4.B) Funzioni `load_elf`:

La funzione OS161 `load_elf` predispone lo spazio degli indirizzi (address space) di un nuovo processo user. In fasi diverse, `load_elf` chiama `load_segment`, `as_prepare_load`, `as_complete_load`, `as_define_region`. Rispondere alle seguenti domande associando la funzione corretta:

1. Quale funzione viene chiamata per allocare lo spazio degli indirizzi nella memoria fisica?
 2. Quale funzione legge il segmento di codice dal file elf?
 3. Quale funzione viene chiamata per impostare gli indirizzi logici e le dimensioni del codice e dei segmenti di dati?
 4. Quale funzione legge il segmento di dati dal file elf?
-

Domanda 5 (OS161 Lab4 - Processi e System Calls)

Contesto: Consider the framework of os161 lab4. Rispondere alle domande seguenti relative a lab4:

Q5.A) Domande a risposta multipla/Vero-Falso:

1. In lab4 un processo deve essere identificato da un pid. Il pid è associato a un processo in fase di bootstrap?
2. Considerare le funzioni `sys_exit` e `sys_waitpid` implementate in lab4. Sono chiamate di sistema che possono essere chiamate dai processi utente?
3. Si chiama `proc_destroy` all'interno della chiamata di sistema `waitpid`?
4. La funzione `common_prog` produce un crash quando mancano implementazione e utilizzo opportuno di `waitpid` (o funzioni correlate, realizzate in lab4)?
5. Considerare le funzioni `sys_exit` e `sys_waitpid` implementate in lab4. Sono già presenti nella versione base di OS161?
6. Si chiama `proc_destroy` all'interno della chiamata di sistema `exit`?

7. In lab4 un processo deve essere identificato da un pid. Un pid è un puntatore?
8. Si chiama `proc_destroy` direttamente dalla funzione `thread_exit`?

SOLUZIONI

Domanda 1 (Memoria Virtuale e Paginazione)

Q1.A) Risposte e Motivazioni:

1. **FALSO.** La presenza di una TLB non garantisce sempre un tempo di accesso più rapido. Il Tempo di Accesso Effettivo (EAT) dipende dall'hit ratio. Nel caso peggiore (0% di hit), il sistema deve prima interrogare (senza successo) la TLB, poi accedere alla Page Table in RAM, e infine accedere al frame in RAM, risultando strettamente più lento di un sistema senza TLB.
2. **VERO.** Un sistema di paginazione permette la condivisione della memoria fisica. Due pagine virtuali appartenenti a processi diversi possono essere mappate allo stesso frame fisico (ad esempio per l'utilizzo di librerie condivise come `.so/.dll`, o segmenti di memoria condivisa per IPC).
3. **FALSO.** Anche se i frame allocati superano la dimensione del working set, il tasso di page fault non scenderà mai matematicamente a zero a causa dei *Compulsory Misses* (o cold misses). Quando un processo inizia, deve inevitabilmente generare page fault per caricare il suo working set iniziale dal disco alla RAM fisica.
4. **VERO.** Questo è il principio alla base delle **Page Table a più livelli** (Multi-level Page Tables) o delle **Inverted Page Tables (IPT)**. In una tabella a due o più livelli, le page table interne vengono allocate in RAM solo se il processo utilizza effettivamente quella specifica porzione di spazio virtuale, riducendo dinamicamente l'overhead di memoria rispetto a una tabella lineare che copre tutto lo spazio.
5. **VERO.** Aumentare la dimensione della pagina incrementa la cosiddetta "TLB Reach" (TLB Reach = Numero di entry TLB $\times$ Dimensione Pagina). Coprendo porzioni più ampie di memoria con la stessa singola entry, si riduce il numero di traduzioni necessarie e si migliora l'efficienza della TLB, compensando la potenziale frammentazione interna.
6. **FALSO.** L'unico algoritmo che garantisce matematicamente il minor numero possibile di page fault è l'algoritmo Ottimo (OPT o di Belady). L'algoritmo LRU (Least Recently Used) è un'ottima approssimazione pratica che garantisce solo di sfrattare la pagina che non è stata usata da più tempo, ma non offre garanzie di ottimalità assoluta.

Q1.B) Esercizio LRU:

1. Calcolo Stringa Riferimenti:

La dimensione della pagina è di 8 KB (2^{13} byte). Pertanto, i 13 bit meno significativi di ogni indirizzo esadecimale costituiscono l'offset. Si ottiene il Numero di Pagina Virtuale (VPN) scartando questi 13 bit (shift a destra di 13 posizioni).

- 1F40 $\rightarrow$ Pagina 0
- 3E20 $\rightarrow$ Pagina 1
- 5D18 $\rightarrow$ Pagina 2
- 7C9E $\rightarrow$ Pagina 3
- 2AF3 $\rightarrow$ Pagina 1
- 6B2C $\rightarrow$ Pagina 3
- 13B2 $\rightarrow$ Pagina 0
- 24AB $\rightarrow$ Pagina 1
- 54BC $\rightarrow$ Pagina 2
- 12AF $\rightarrow$ Pagina 0

Stringa dei riferimenti (VPN): **0, 1, 2, 3, 1, 3, 0, 1, 2, 0**

2. Tracciamento Frame e Page Fault (LRU):

Simulazione con 3 frame fisici disponibili. La politica Least Recently Used (LRU) seleziona come vittima la pagina che non è stata utilizzata da più tempo all'indietro nella stringa dei riferimenti.

Riferimento (VPN)	0	1	2	3	1	3	0	1	2	0
Frame 1	0	0	0	3	3	3	3	3	2	2
Frame 2	-	1	1	1	1	1	1	1	1	1
Frame 3	-	-	2	2	2	2	0	0	0	0
Esito (H/F)	F	F	F	F	H	H	F	H	F	H

Tabella 1: H = Hit (successo), F = Page Fault (errore)

3. Numero totale di Page Fault:

Come illustrato nella tabella di simulazione, si verificano un totale di **6 Page Fault** (in corrispondenza delle richieste per le pagine 0, 1, 2, 3, 0 e 2). I primi tre sono fault obbligatori (*compulsory misses*) necessari per riempire i frame inizialmente vuoti.

Domanda 2 (File System e Inodes)

Q2.A) Dimensione massima del file:

Per calcolare la dimensione massima, determiniamo innanzitutto la capacità strutturale dell'inode. Sapendo che la dimensione del blocco è 4 KB (4096 byte) e un puntatore occupa 4 byte, ogni blocco indice può contenere $4096/4 = 1024$ puntatori.

- **Direct pointers:** 12 puntatori diretti $\rightarrow 12 \times 4 \text{ KB} = \mathbf{48 \text{ KB}}$
- **Single indirect:** 1 blocco indice (1024 puntatori) $\rightarrow 1024 \times 4 \text{ KB} = \mathbf{4 \text{ MB}}$
- **Double indirect:** 1 blocco indice $\rightarrow 1024$ blocchi indice $\rightarrow 1024^2$ puntatori $\rightarrow 1.048.576 \times 4 \text{ KB} = \mathbf{4 \text{ GB}}$
- **Triple indirect:** 1024^3 puntatori $\rightarrow 1.073.741.824 \times 4 \text{ KB} = \mathbf{4 \text{ TB}}$

Capacità strutturale totale = 4 TB + 4 GB + 4 MB + 48 KB.

Conclusione per l'esame:

Sebbene l'architettura dell'inode supporti teoricamente file di dimensioni leggermente superiori a 4 TB, il testo specifica che la partizione del disco riservata ai blocchi dati è limitata a 1 TB. Di conseguenza, la dimensione massima effettiva di un singolo file in questo specifico file system è vincolata dal limite fisico hardware, ovvero **1 TB**.

Q2.B) Blocchi Dati, Blocchi Indice e Frammentazione:

Dati: Dimensione file = 25.600 KB. Dimensione blocco = 4 KB.

- **Blocchi Dati e Frammentazione:** Il numero totale di blocchi dati si ottiene dividendo la dimensione del file per la dimensione del blocco:

$$25.600 \text{ KB} / 4 \text{ KB} = \mathbf{6.400 \text{ blocchi dati}}$$

Poiché la divisione restituisce un numero intero esatto, il file riempie perfettamente i blocchi allocati. Di conseguenza, la **frammentazione interna è esattamente 0 byte**.

- **Distribuzione nei puntatori:**

- **Direct:** I primi 12 blocchi sono allocati direttamente. Restano $6400 - 12 = 6388$ blocchi.
- **Single Indirect:** Utilizza 1 blocco indice per mappare i successivi 1024 blocchi. Restano $6388 - 1024 = 5364$ blocchi.
- **Double Indirect:** Deve gestire i restanti 5364 blocchi. Poiché ogni blocco indice di primo livello può puntare a 1024 blocchi dati, occorrono $\lceil 5364/1024 \rceil = 6$ blocchi indice intermedi (di cui 5 completamente pieni e 1 riempito parzialmente con 244 puntatori).
- **Calcolo Blocchi Indice Totali:** Sommando i blocchi indice necessari per supportare questa struttura:
 - 1 blocco per il Single Indirect.
 - 1 blocco "radice" (root) per il Double Indirect.
 - 6 blocchi intermedi all'interno della struttura Double Indirect.
 - **Totale = 8 blocchi indice.**

Q2.C) Calcolo del Logical Block Number:

Dati: Dimensione file = 10 GB. Offset = -1 MB dalla fine. Dimensione blocco = 4 KB.

- **Calcolo della Posizione Target:** Convertiamo la dimensione totale del file in Megabyte (MB) per semplificare la sottrazione:

$$10 \text{ GB} = 10 \times 1024 \text{ MB} = 10.240 \text{ MB}$$

La posizione di destinazione all'interno del file è:

$$\text{Target} = 10.240 \text{ MB} - 1 \text{ MB} = 10.239 \text{ MB}$$

- **Conversione e Calcolo del Blocco:** Convertiamo la posizione target in Kilobyte (KB) in modo da avere la stessa unità di misura del blocco:

$$10.239 \text{ MB} \times 1024 \text{ KB/MB} = 10.484.736 \text{ KB}$$

Il numero del blocco logico (calcolato a partire da 0) si ottiene dividendo la posizione target per la dimensione del blocco:

$$\text{Logical Block Number} = \left\lfloor \frac{10.484.736 \text{ KB}}{4 \text{ KB/blocco}} \right\rfloor = \mathbf{2.621.184}$$

Poiché il risultato è un numero intero esatto, il puntatore viene posizionato esattamente al primo byte (offset interno 0) del blocco logico numero 2.621.184.

Domanda 3 (I/O, HDD, SSD e RAID)

Q3.A) Risposte e Motivazioni:

1. **FALSO.** Sebbene il DMA sia eccellente per trasferimenti di blocchi di grandi dimensioni, non è efficiente per trasferimenti molto piccoli (es. un singolo byte da tastiera). L'overhead richiesto alla CPU per impostare i registri del controller DMA (indirizzo sorgente, destinazione, contatore) richiederebbe più cicli di clock rispetto all'esecuzione diretta dell'I/O tramite un semplice interrupt.

2. **VERO.** In ambienti di rete ad altissima velocità, l'arrivo continuo di pacchetti genera una tempesta di interrupt ("interrupt storm" o livelock). La CPU spenderebbe tutto il suo tempo a fare context switch per gestire gli interrupt, azzerando il throughput utile. Il polling puro (o un approccio ibrido) permette invece alla CPU di processare i pacchetti a lotti (batching) in modo molto più efficiente, eliminando l'overhead del context switch.
3. **FALSO.** Nonostante le SSD siano generalmente molto più veloci, non lo sono *sempre*. Le SSD soffrono del problema dell'*erase-before-write* (Write Amplification). Se un'operazione di scrittura avviene su un blocco che deve essere prima cancellato, o se l'SSD sta eseguendo la Garbage Collection in background, la latenza di quella singola operazione può subire picchi altissimi, superando in quel preciso istante la latenza di una scrittura sequenziale su HDD.
4. **FALSO.** Gli HDD subiscono gravi cali di prestazioni a causa della frammentazione perché i dati sparsi costringono la testina a continui spostamenti meccanici (Seek Time e Rotational Latency). Le SSD non hanno parti meccaniche in movimento; l'accesso a indirizzi logici non contigui avviene con la stessa latenza dell'accesso sequenziale (accesso puramente elettronico).
5. **VERO.** Il RAID 5 soffre della penalità delle scritture di piccole dimensioni ("small write penalty" o Read-Modify-Write), poiché deve ricalcolare la parità leggendo i vecchi dati. Uno scheduler I/O ottimizzato può ritardare e riordinare le scritture accorpandole in una "full-stripe write" (scrittura dell'intera striscia in un solo colpo), eliminando del tutto la necessità di pre-leggere i vecchi blocchi e la vecchia parità, migliorando drasticamente le prestazioni.
6. **FALSO.** Il RAID 0 (Striping) divide i dati parallelamente sui dischi per massimizzare le prestazioni, ma **non offre alcuna ridondanza** (nessuna parità o mirroring). Di conseguenza, l'affidabilità diminuisce drasticamente rispetto a un disco singolo: la rottura di un solo disco all'interno dell'array RAID 0 causa la perdita irreversibile di tutti i dati presenti nell'intero sistema.

Domanda 4 (Indirizzamento OS161 e load_elf)

Q4.A) Traduzione da Indirizzi Fisici a Logici:

Definizione dei Limiti Fisici di P1 (Page Size = 4 KB = 0x1000):

- **Segmento 1 (Base1):** 3 pagine → Dimensione 0x3000. Range Fisico valido: da 0x200000 a 0x202FFF. Base Virtuale: 0x4000.
- **Segmento 2 (Base2):** 4 pagine → Dimensione 0x4000. Range Fisico valido: da 0x100000 a 0x103FFF. Base Virtuale: 0x8000.

Regola Kernel MIPS: Ogni indirizzo fisico viene mappato nello spazio logico del kernel (kseg0) semplicemente sommando l'offset 0x80000000.

1. **Indirizzo Fisico:** 0x100A00
Kernel Logical: $0x80000000 + 0x100A00 = 0x80100A00$.
User Logical: L'indirizzo rientra nel Segmento 2 (Base2).
 Offset: $0x100A00 - 0x100000 = 0xA00$.
 Indirizzo Utente = $0x8000 + 0xA00 = 0x8A00$.
2. **Indirizzo Fisico:** 0x10A0F0
Kernel Logical: $0x80000000 + 0x10A0F0 = 0x8010A0F0$.
User Logical: L'indirizzo supera il limite superiore del Segmento 2 (0x103FFF). **Non appartiene a P1** (Out of bounds).

3. **Indirizzo Fisico:** 0x206500

Kernel Logical: $0x80000000 + 0x206500 = 0x80206500$.

User Logical: L'indirizzo supera il limite superiore del Segmento 1 (0x202FFF). **Non appartiene a P1** (Out of bounds).

4. **Indirizzo Fisico:** 0x202D00

Kernel Logical: $0x80000000 + 0x202D00 = 0x80202D00$.

User Logical: L'indirizzo rientra nel Segmento 1 (Base1).

Offset: $0x202D00 - 0x200000 = 0x2D00$.

Indirizzo Utente = $0x4000 + 0x2D00 = 0x6D00$.

Q4.B) Funzioni load_elf:

La funzione `load_elf` esegue il parsing del file eseguibile e orchestra la creazione del processo in memoria seguendo un ordine preciso.

1. **Quale funzione viene chiamata per allocare lo spazio degli indirizzi nella memoria fisica?**

Risposta: `as_prepare_load`. Dopo aver definito tutte le regioni, questa funzione alloca effettivamente i frame fisici in RAM necessari per ospitare il processo (in `dumbvm`, chiama `getppages`).

2. **Quale funzione legge il segmento di codice dal file elf?**

Risposta: `load_segment`. Questa funzione è responsabile del trasferimento fisico dei byte dal file su disco (tramite `vnode`) alla memoria RAM allocata.

3. **Quale funzione viene chiamata per impostare gli indirizzi logici e le dimensioni del codice e dei segmenti di dati?**

Risposta: `as_define_region`. Viene chiamata in un ciclo iniziale per ogni Program Header (segmento) per registrare all'interno della struttura `addrspace` l'indirizzo virtuale di partenza (`VStart`) e la sua dimensione.

4. **Quale funzione legge il segmento di dati dal file elf?**

Risposta: `load_segment`. *Attenzione:* `load_segment` è una funzione generica. Il loader ELF non fa distinzione tra codice e dati al momento del caricamento; utilizza la stessa identica funzione per trasferire qualsiasi segmento definito come "loadable" dal disco alla memoria. (`as_complete_load` viene invece chiamata alla fine di tutto il processo, solitamente per impostare i permessi di sola lettura sul segmento di codice appena caricato).

Domanda 5 (OS161 Lab4 Processi e System Calls)

Q5.A) Domande a risposta multipla/Vero-Falso:

1. **FALSO.** Il sistema di gestione dei PID viene inizializzato durante il bootstrap, ma un PID specifico viene generato e associato a un processo solo dinamicamente al momento della sua creazione (ad esempio tramite `fork` o `runprogram`).
2. **VERO.** (Dal punto di vista funzionale). `sys_exit` e `sys_waitpid` sono le routine kernel sviluppate appositamente per servire le omonime system call invocate dai processi utente. (Tecnicamente, il processo utente chiama il wrapper di libreria, generando una trap che fa invocare la funzione `sys_*` dal dispatcher del kernel).
3. **VERO.** Quando un processo termina diventa uno "zombie". È responsabilità del processo padre, all'interno della system call `waitpid`, recuperare lo status di terminazione e chiamare infine `proc_destroy` per deallocare in sicurezza la struttura del figlio.

4. **VERO.** `common_prog` lancia i programmi dal menu del kernel. Se non utilizza `waitpid` per attendere la terminazione del processo utente, il menu riappare immediatamente. Un successivo spegnimento del sistema causerebbe un kernel panic a causa dei processi utente ancora attivi in background.
5. **FALSO.** Nella versione base di OS161 (prima del Lab 4) queste chiamate non sono implementate. Invocandole, il kernel restituisce semplicemente il codice di errore `ENOSYS` (System call non implementata).
6. **FALSO.** Se un processo chiamasse `proc_destroy` su se stesso all'interno di `exit`, distruggerebbe la struttura contenente il proprio valore di ritorno, rendendo impossibile per il padre recuperarlo. Inoltre, andrebbe a deallocare la memoria (incluso lo stack) su cui il thread corrente sta ancora eseguendo, causando un crash fatale.
7. **FALSO.** Il PID (Process ID) è un valore intero (tipicamente definito col tipo `pid_t`), non un puntatore. È un identificatore numerico astratto che il kernel utilizza internamente, spesso come indice di un array (la *process table*), per recuperare in modo sicuro il puntatore alla vera struttura dati `struct proc` associata al processo.
8. **FALSO.** Esattamente per lo stesso motivo analizzato nella risposta 6. `thread_exit` gestisce lo spegnimento a basso livello del thread in esecuzione. Se chiamasse `proc_destroy`, deallocherebbe immediatamente le strutture vitali del processo e il relativo stato di uscita. Il processo deve invece sopravvivere come "zombie" affinché il processo padre possa raccogliergli l'esito tramite `waitpid`, la quale sarà poi l'unica responsabile della chiamata a `proc_destroy`.